

Q.1 Create table employee.

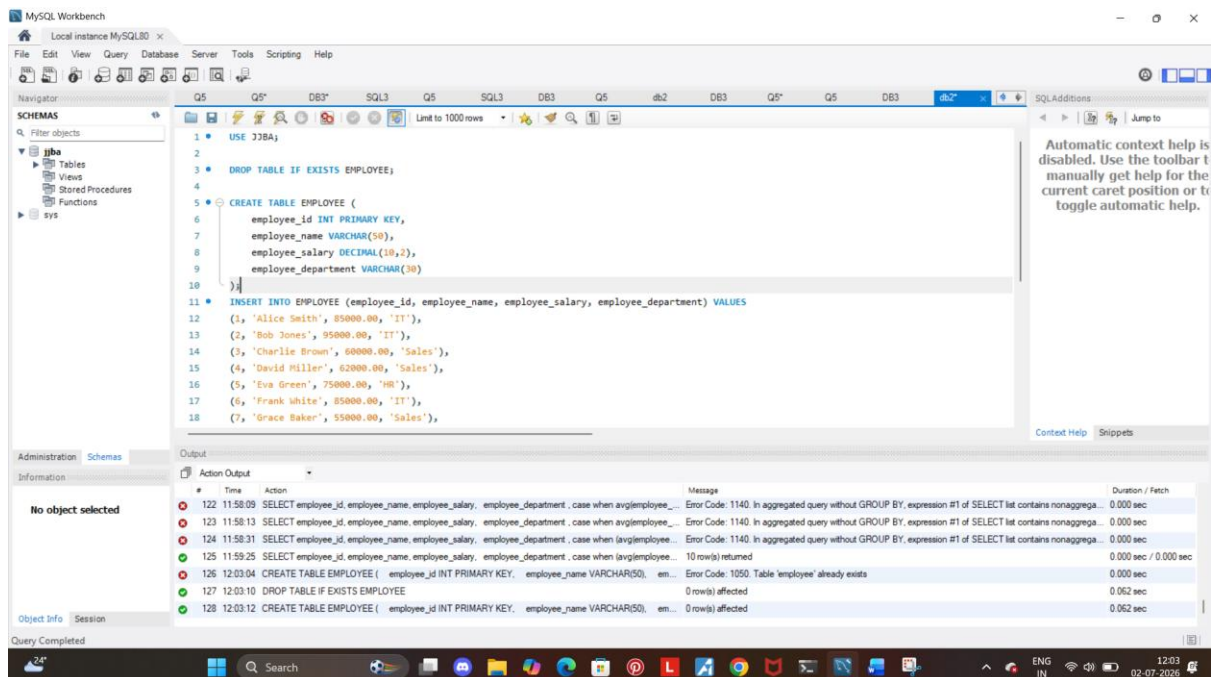
Query:

USE JJBA;

DROP TABLE IF EXISTS EMPLOYEE;

```
CREATE TABLE EMPLOYEE (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(50),  
    employee_salary DECIMAL(10,2),  
    employee_department VARCHAR(30)  
);
```

Screenshot:



Q. 2 Insert data of 10 employees.

Query:

```
USE JJBA;
```

```
DROP TABLE IF EXISTS EMPLOYEE;
```

```
CREATE TABLE EMPLOYEE (
```

```
    employee_id INT PRIMARY KEY,
```

```
    employee_name VARCHAR(50),
```

```
    employee_salary DECIMAL(10,2),
```

```
    employee_department VARCHAR(30)
```

```
);
```

```
INSERT INTO EMPLOYEE (employee_id, employee_name, employee_salary, employee_department) VALUES
```

```
(1, 'Alice Smith', 85000.00, 'IT'),
```

```
(2, 'Bob Jones', 95000.00, 'IT'),
```

```
(3, 'Charlie Brown', 60000.00, 'Sales'),
```

```
(4, 'David Miller', 62000.00, 'Sales'),
```

```
(5, 'Eva Green', 75000.00, 'HR'),
```

```
(6, 'Frank White', 85000.00, 'IT'),
```

```
(7, 'Grace Baker', 55000.00, 'Sales'),
```

```
(8, 'Henry Clark', 90000.00, 'IT'),
```

```
(9, 'Ivy Davis', 72000.00, 'HR'),
```

```
(10, 'Jack Wilson', 62000.00, 'Sales');
```

Screenshot:

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

SCHEMAS

- Filter objects
- Tables
- Views
- Stored Procedures
- Functions
- sys

Q5 Q5* DB3* SQL3 Q5 SQL3 DB3 Q5 db2 DB3 Q5* Q5 DB3 db2

Limit to 1000 rows

SQL editor

```
18 (7, 'Grace Baker', 55000.00, 'Sales'),
19 (8, 'Henry Clark', 90000.00, 'IT'),
20 (9, 'Ivy Davis', 72000.00, 'HR'),
21 (10, 'Jack Wilson', 62000.00, 'Sales');
22 • select * from employee;
23
24 • SELECT employee_id,
25 employee_name,
26 employee_salary,
27 employee_department ,
28 case
```

Result Grid

	employee_id	employee_name	employee_salary	employee_department
1	Alice Smith	85000.00	IT	
2	Bob Jones	95000.00	IT	
3	Charlie Brown	60000.00	Sales	
4	David Miller	62000.00	Sales	
5	Eva Green	75000.00	HR	

Output

Information

No object selected

Object Info Session

Query Completed

Output

#	Time	Action	Message	Duration / Fetch
124	11:58:31	SELECT employee_id, employee_name, employee_salary, employee_department, case when (avg(employee...	Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains nonaggrega...	0.000 sec
125	11:59:25	SELECT employee_id, employee_name, employee_salary, employee_department, case when (avg(employee...	10 row(s) returned	0.000 sec / 0.000 sec
126	12:03:04	CREATE TABLE EMPLOYEE (employee_id INT PRIMARY KEY, employee_name VARCHAR(50), em...	Error Code: 1050. Table 'employee' already exists	0.000 sec
127	12:03:10	DROP TABLE IF EXISTS EMPLOYEE	0 row(s) affected	0.062 sec
128	12:03:12	CREATE TABLE EMPLOYEE (employee_id INT PRIMARY KEY, employee_name VARCHAR(50), em...	0 row(s) affected	0.062 sec
129	12:04:40	INSERT INTO EMPLOYEE (employee_id, employee_name, employee_salary, employee_department) VALUES...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec
130	12:04:48	select * from employee LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

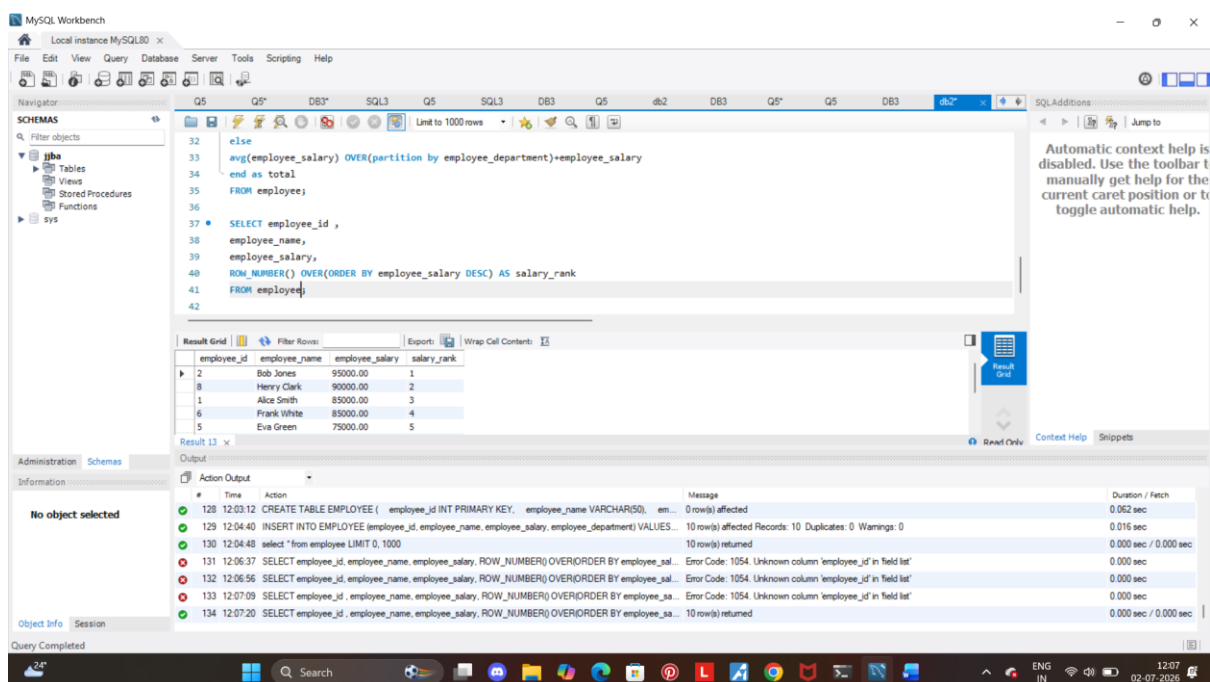
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Q.3 Rank the employees based on their salary.

Query:

```
SELECT employee_id ,  
employee_name,  
employee_salary,  
ROW_NUMBER() OVER(ORDER BY employee_salary DESC) AS salary_rank  
FROM employee;
```

Screenshot:



Insights:

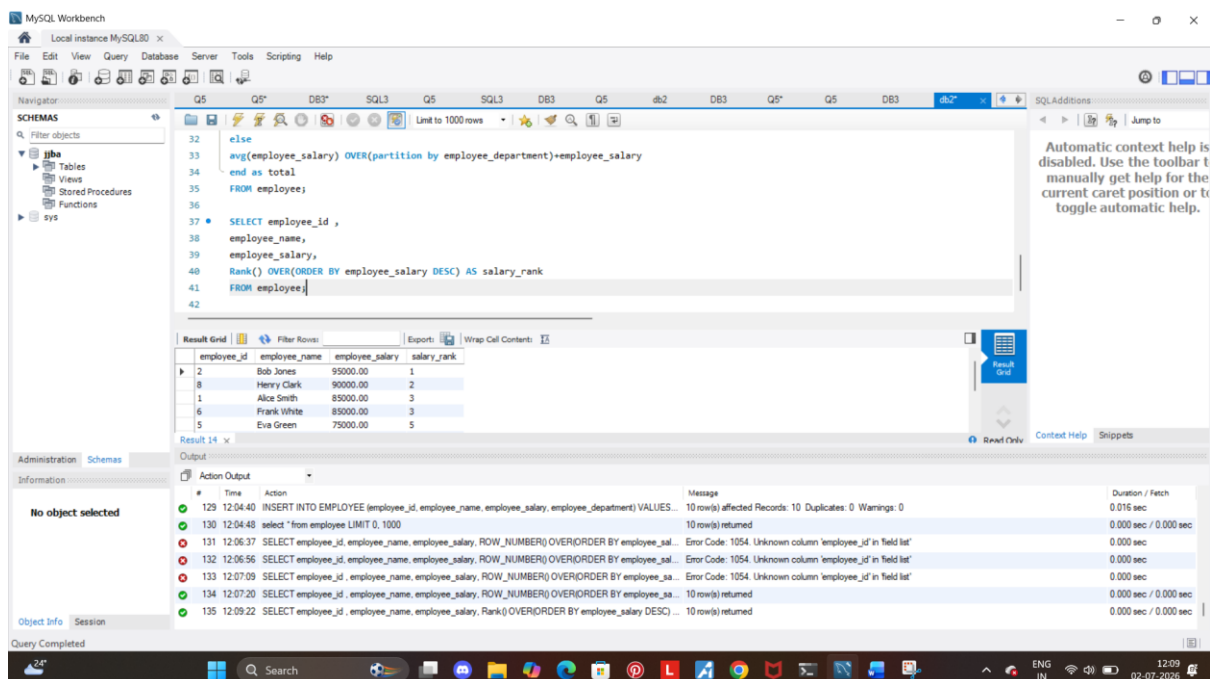
- The row_number function assignment the rank to the employees.
- The employee_salary was grouped in a descending order and the rank was assigned to them just like that.
- The row_number function only assigns numbers that is 1,2,etc.. so it does not assign same rank to employees having same salary.

Q.3 Assign rank to employees based on the data.

Query:

```
SELECT employee_id ,  
employee_name,  
employee_salary,  
Rank() OVER(ORDER BY employee_salary DESC) AS salary_rank  
FROM employee;
```

Screenshot:



Insights:

- The rank function actually works on the data.
- Based on it the function assigns the rank to the employees.
- Hence, employees having same salary are ranked same but it skips the next number that is after ranking 2 employees as 'rank 2' it directly jumps on 5 instead of 4.

Q. 5 Rank the employees based on salary:

Query:

```
SELECT employee_id ,  
employee_name,  
employee_salary,  
dense_Rank() OVER(ORDER BY employee_salary DESC) AS salary_rank  
FROM employee;
```

Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL editor contains a query to rank employees by salary. The query is as follows:

```
32 else  
33 avg(employee_salary) OVER(partition by employee_department)+employee_salary  
34 end as total  
35 FROM employee;  
36  
37 • SELECT employee_id ,  
38 employee_name,  
39 employee_salary,  
40 dense_Rank() OVER(ORDER BY employee_salary DESC) AS salary_rank  
41 FROM employee;  
42
```

The Results tab shows the output of the query, displaying a table with the following data:

employee_id	employee_name	employee_salary	salary_rank
2	Bob Jones	95000.00	1
8	Henry Clark	90000.00	2
1	Alice Smith	85000.00	3
6	Frank White	85000.00	3
5	Eva Green	75000.00	4

The Output tab shows the execution log, indicating that the query was executed successfully and returned 10 rows.

Insights:

- The problem with rank function is over come by the dense_rank function.
- It properly ranks the employees even if they have same salary, and after ranking employees with same salary it doesn't skip the next rank like rank function does.
- Instead it continues ranking sequentially.

Q. 6 Calculate the average salary of employees, by grouping the employees with same salary one after another, if the actual salary of employee in a particular department is greater than average salary minus the average salary from the actual salary and if not add the actual salary in the average salary.

Query:

```
SELECT employee_id,  
employee_name,  
employee_salary,  
employee_department ,  
case  
when (avg(employee_salary) OVER(partition by employee_department) > employee_salary)  
then  
avg(employee_salary) OVER(partition by employee_department)-employee_salary  
else  
avg(employee_salary) OVER(partition by employee_department)+employee_salary  
end as total  
FROM employee;
```


Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL editor contains a query that calculates the average salary for each department and then adds and subtracts the employee's salary to find the total deviation from the department average. The query is as follows:

```
26 employee_salary,  
27 employee_department ,  
28 case  
29 when (avg(employee_salary) OVER(partition by employee_department) > employee_salary)  
30 then  
31 avg(employee_salary) OVER(partition by employee_department)-employee_salary  
32 else  
33 avg(employee_salary) OVER(partition by employee_department)+employee_salary  
34 end as total  
35 FROM employee_salary  
36
```

The Results grid shows the following data:

employee_id	employee_name	employee_salary	employee_department	total
5	Eva Green	75000.00	HR	148500.000000
9	Ivy Davis	72000.00	HR	1500.000000
1	Alice Smith	85000.00	IT	3750.000000
2	Bob Jones	95000.00	IT	163750.000000
6	Frank White	85000.00	IT	3750.000000

The bottom panel shows the execution log with the following entries:

Time	Action	Message	Duration / Fetch
131 12:06:37	SELECT employee_id, employee_name, employee_salary, ROW_NUMBER() OVER(ORDER BY employee_sal...	Error Code: 1054. Unknown column 'employee_id' in field list	0.000 sec
132 12:06:56	SELECT employee_id, employee_name, employee_salary, ROW_NUMBER() OVER(ORDER BY employee_sal...	Error Code: 1054. Unknown column 'employee_id' in field list	0.000 sec
133 12:07:09	SELECT employee_id, employee_name, employee_salary, ROW_NUMBER() OVER(ORDER BY employee_sa...	Error Code: 1054. Unknown column 'employee_id' in field list	0.000 sec
134 12:07:20	SELECT employee_id, employee_name, employee_salary, ROW_NUMBER() OVER(ORDER BY employee_sa...	10 row(s) returned	0.000 sec / 0.000 sec
135 12:09:22	SELECT employee_id, employee_name, employee_salary, Rank() OVER(ORDER BY employee_salary DESC)	10 row(s) returned	0.000 sec / 0.000 sec
136 12:15:44	SELECT employee_id, employee_name, employee_salary, dense_Rank() OVER(ORDER BY employee_sal...	10 row(s) returned	0.016 sec / 0.000 sec
137 12:23:50	SELECT employee_id, employee_name, employee_salary, employee_department, case when (avg(employee...	10 row(s) returned	0.000 sec / 0.000 sec

Insights:

- Here the average of HR department is 72,000 and after adding and subtracting it gives the output in the total column.

Q.3

Query: